LLMs and Formal Methods: A survey

April 3, 2026

Contents

1 Scope and structure 1

2 Taxonomy 1

3 Surveys and roadmaps 2

4 Specification languages and requirements 2

5 Verified code generation and proof synthesis for code 3

6 AI-assisted mechanised proving 4

7 Benchmarks and datasets 7

8 Adjacent specification and AI-native coding systems 7

Bibliography 7

1 Scope and structure

This survey is organised around the two-way intersection between large language models and formal methods: *formal methods for constraining or checking AI-assisted software development*, and *LLMs for lowering the cost of formal specification, verification, and mechanised proof*. After a short section of surveys and roadmaps, the material is organised directly by technical category, combining papers, systems, benchmarks, and industry artefacts where that gives the clearest picture of the space. The main emphasis is on systems with a direct formal-methods component, but the survey also notes adjacent specification and programming artefacts when they illuminate likely interfaces between natural-language intent, intermediate specifications, AI-native implementation languages, and verifier-oriented workflows.

2 Taxonomy

Area	What sits here	Section
Surveys and roadmaps	High-level surveys, position papers, and roadmaps that frame the field and its major research directions.	§3
Specification languages and requirements	Natural language to logic/specification, intent formalisation, and auditable requirement capture.	§4
Verified code generation and proof synthesis for code	Verifier-in-the-loop generation of code, contracts, assertions, and proofs for Dafny, Verus, ACSL, SPARK, and related systems.	§5
AI-assisted mechanised proving	Interactive theorem proving, premise selection, tactic generation, whole-proof synthesis, and proof repair in Lean, Isabelle, Rocq/Coq, and Metamath.	§6
Benchmarks and datasets	Shared evaluation sets for theorem proving, verifiable code generation, and repository-scale proof.	§7
Adjacent specification and AI-native coding systems	Spec-centric languages, AI-oriented implementation languages, and spec-driven development toolchains that are not themselves formal methods results but may shape how intent is structured before verification.	§8

3 Surveys and roadmaps

- **Autoformalisation survey**. [1]
- **Survey on LLMs for software-requirement formalisation**. [2]
- **Survey on deep learning for theorem proving**. [3]
- **Roadmap on LLMs and formal methods for trustworthy agents**. [4]
- **Formal mathematical reasoning as an AI frontier**. [5]
- **Two-way roadmap between requirements engineering and LLMs**. [6]

4 Specification languages and requirements

- **VERIFAI**. Early work on traceability and verification of natural-language requirements, useful as background for NL-to-spec workflows. [7]
- **Formal Specifications from Natural Language**. Direct NL-to-formal-spec framing paper. [8]
- **Autoformalization with Large Language Models**. Early LLM-based autoformalisation of informal mathematical text. [9]

- **Process-Driven Autoformalization in Lean 4**. Systematic pipeline for translating informal mathematical text into Lean 4 proofs with process supervision. [10]
- **Improving Autoformalization using Type Checking**. Shows that type-checking feedback significantly reduces autoformalization errors. [11]
- **nl2spec**. Interactive translation from unstructured requirements to temporal logic with editable sub-translations. [12]
- **NL2TL**. A general framework for mapping natural-language instructions to temporal logic across multiple domains. [13]
- **SpecGen**. Automated generation of formal specifications from natural-language requirements. [14]
- **Specify What?** and **Seeking Specifications**. Neuro-symbolic work on deriving and refining specifications from informal descriptions. [15, 16]
- **Doc2Spec**. A multi-agent approach that induces formal specifications from natural-language documents. [17]
- **Trustworthy Formal Natural Language Specifications**. A proof-assistant-native route to formalising natural-language specifications inside Lean. [18]
- **Microsoft intent formalisation agenda**. Industry framing that treats user-intent formalisation as a central reliability problem for AI-assisted coding. [19]
- **nl2postcond** and follow-on evaluation work. Microsoft-led efforts on turning programmer intent into checkable postconditions and evaluating this route for verification-aware languages. [20, 21]
- **Supporting Software Formal Verification with Large Language Models: An Experimental Study**. Broader evidence on where LLM assistance helps in formal-specification and verification workflows. [22]

5 Verified code generation and proof synthesis for code

- **DafnySynth / dafny-annotator**. LLM-assisted synthesis of Dafny assertions, invariants, and proof-oriented annotations. [23]
- **AI-assisted Dafny synthesis (Misu et al.)**. Systematic study of LLM-assisted synthesis of verified Dafny methods, evaluating multiple LLMs on generation and annotation tasks at FSE 2024. [24]
- **Laurel**. Uses LLMs to unblock automated verification by synthesising useful proof and specification steps. [25]
- **Verification with Ada/SPARK**. Evaluation of LLM-generated code in a SPARK-based verification setting. [26]
- **LLM code generation with formal specifications and reactive synthesis**. Couples code generation with formal specifications and synthesis constraints. [27]
- **VeCoGen**. Formally verified C generation using LLMs plus a verifier-guided refinement loop. [28]

- **Automatic generation of formal specification and verification annotations using LLMs and test oracles**. Annotation-generation workflow for verification tools. [29]
- **Evaluating LLM-generated ACSL annotations**. Assessment of annotation quality for Frama-C-style verification workflows. [30]
- **AlphaVerus**. Self-improving verified code generation for Verus/Rust. [31]
- **RAG-Verus**. Repository-level proof synthesis for Verus with retrieval augmentation and repository context. [32]
- **Microsoft verus-proof-synthesis**. Umbrella repository for Microsoft's Verus verification automation stack. [33]
- **AutoVerus**. Automated proof generation for Rust/Verus. [34]
- **VeruSAGE**. Agent-based verification for repository-scale Rust systems. [35]
- **VeriStruct**. Planner-based verification of Verus data-structure modules. [36]
- **Generating Safety-Critical Automotive C-programs using LLMs with Formal Verification**. Joint industry-academic evaluation in a safety-critical domain. [37]
- **Clover**. Closed-loop verifiable code generation: LLM synthesises Dafny programs while a separate verifier loop checks and repairs annotations. [38]
- **SAFE**. Self-evolution approach to automated Rust/Verus proof generation, using iterative fine-tuning on successful proof traces. [39]
- **Enchanting**. Uses LLMs combined with static analysis and program verification to synthesise formal specifications from code. [40]
- **Rango**. Adaptive retrieval-augmented proof generation for Coq/Rocq: retrieves similar verified proofs to guide LLM-based proof synthesis. [41]
- **Neural Synthesis for SMT-Assisted Proof-Oriented Programming**. Fine-tunes LLMs to generate F* proof obligations dischargeable by an SMT solver, targeting proof-oriented language verification. [42]
- **Lemur**. Integrates LLMs into automated program verification pipelines (Dafny/Frama-C), combining deductive verification with LLM-generated invariants and annotations. [43]
- **Finding Inductive Loop Invariants using LLMs**. Uses LLMs to synthesise candidate loop invariants, verified by Frama-C/Boogie, with iterative feedback. [44]
- **Evaluating LLMs for VeriFast specifications**. Evaluates the ability of LLMs to generate verifiable memory-safety specifications in VeriFast for C programs. [45]
- **Can LLMs Enable Verification in Mainstream Programming?** Empirical study of LLM-assisted deductive verification across mainstream languages (Java/JML, C/Frama-C), assessing annotation correctness. [46]

6 AI-assisted mechanised proving

- **GPT-f**. Early neural theorem-proving baseline for proof search guidance. [47]
- **Draft, Sketch, and Prove**. Guides formal theorem provers using informal proof sketches. [48]

- **ProofNet**. Autoformalisation and formal proving for undergraduate-level mathematics. [49]
- **Baldur**. Whole-proof generation and repair with large language models. [50]
- **LeanDojo / ReProver**. Open tooling and datasets for Lean-based proving with retrieval-augmented models. [51]
- **COPRA**. In-context agentic theorem proving for interactive assistants. [52]
- **DeepSeek-Prover**. Open Lean prover line with synthetic-data training; the public repository tracks released models. [53, 54]
- **SubgoalXL**. Subgoal-based expert learning for theorem proving. [55]
- **LEGO-Prover**. Neural theorem proving with growing libraries. [56]
- **Lean Copilot**. LLM inference inside Lean for tactic suggestion, proof search, and premise selection. [57]
- **APOLLO**. Lean+LLM collaboration driven by compiler-guided repair. [58]
- **HybridProver**. Combines theorem proving with LLM-driven proof synthesis and repair. [59]
- **A Minimalist Proof Language for Neural Theorem Proving over Isabelle/HOL**. Proof-language design for neural theorem proving over Isabelle/HOL. [60]
- **Agentic Program Verification / AutoRocq**. Agentic Rocq/Coq verification via iterative prover feedback. [61]
- **AutoReal on seL4**. LLM-driven theorem proving in an industrial-scale verification setting. [62]
- **Vibe Coding an LLM-powered Theorem Prover / Isabellm**. Lightweight Isabelle/HOL proving with both a paper and an open implementation. [63, 64]
- **LeanNavigator**. Large-scale generation of additional Lean theorems and proofs from Mathlib4. [65]
- **AlphaProof programme**. Google DeepMind's formal mathematical reasoning effort, framed publicly through reinforcement learning and formal proof search. [66, 67]
- **Harmonic / Aristotle**. Industry effort on olympiad-level automated theorem proving and formal mathematical reasoning. [68]
- **formal-conjectures**. Repository of Lean-formalised conjectures intended to support formal reasoning research. [69]
- **TheoremLlama**. Fine-tunes general-purpose LLMs on Lean 4 proof data to create Lean-expert models. [70]
- **FVEL**. Interactive formal verification environment coupling LLMs with Isabelle/HOL for software property proving. [71]
- **Selene**. Pioneer work on automated proof generation for software verification, bridging LLMs with Lean for repository-scale proofs. [72]
- **HyperTree Proof Search**. Tree-search algorithm for neural theorem proving in Lean and Metamath, with online training via MCTS-style exploration. [73]
- **Lean-STaR**. Trains LLMs to interleave informal thinking steps with Lean proof steps, improving proof generation quality. [74]

- **REFACTOR**. Learns to identify and extract reusable lemmas from existing Lean proofs to grow the library. [75]
- **Graph2Tac**. Graph neural network approach to premise selection and tactic prediction for Lean/Coq. [76]
- **Formal Mathematics Statement Curriculum Learning**. Uses expert iteration and curriculum learning to train LLMs for formal mathematics in Lean and Metamath. [77]
- **Llemma**. Open language model specialised for mathematics, pre-trained on formal and informal mathematical text; widely used as a backbone for theorem-proving models. [78]
- **Goedel-Prover** and **Goedel-Prover-V2**. Frontier open-source Lean provers trained on large synthetic datasets, with V2 adding scaffolded data synthesis and self-correction. [79, 80]
- **DeepSeek-Prover-V2**. Advances Lean theorem proving with reinforcement learning over subgoal decomposition trees. [81]
- **Kimina-Prover**. Large formal reasoning model trained with RL, achieving strong miniF2F results. [82]
- **AlphaGeometry**. Neurosymbolic system that solves olympiad geometry problems at silver-medal level using a language model paired with a symbolic geometry engine. [83]
- **Autoformalizing Euclidean Geometry**. Demonstrates autoformalization of Euclidean geometry into Lean, translating informal geometric statements and proofs. [84]
- **LINC** and **Logic-LM** and **SatLM**. Neurosymbolic approaches that combine LLMs with first-order logic provers, symbolic solvers, and SAT/SMT reasoners respectively to achieve faithful formal reasoning. [85, 86, 87]
- **Proof Artifact Co-Training (PACT)**. Jointly trains a language model on proof artifacts (tactic states, intermediate goals) alongside informal text, improving formal proof generation across interactive proof assistants. [88]
- **Lyra**. Orchestrates dual correction in automated theorem proving: an LLM generates proof drafts while a separate refinement loop corrects both the informal reasoning and formal proof steps. [89]
- **Subgoal-based Demonstration Learning**. ICML 2024 approach that decomposes formal proofs into subgoals and uses demonstration learning to train LLMs on structured subgoal sequences for Lean theorem proving. [90]
- **HunyuanProver**. Scalable data synthesis framework combined with guided tree search for automated Lean theorem proving, achieving strong miniF2F results. [91]
- **Proving Theorems Recursively**. Decomposes complex theorems into simpler subgoals recursively, using LLMs to prove each subgoal independently and assemble the full proof. [92]
- **BFS-Prover**. Scalable best-first tree search for LLM-based automatic theorem proving in Lean, with online value estimation guiding proof search. [93]
- **InternLM2.5-StepProver**. Advances automated Lean theorem proving via expert iteration on large-scale problem sets, training step-level models with iterative data synthesis. [94]

7 Benchmarks and datasets

- **miniF2F**, **PUTNAMBENCH**, and **FormalMATH**. Benchmarks for formal mathematical reasoning and theorem proving across systems and domains. [95, 96, 97]
- **FIMO**. Challenge dataset for automated theorem proving. [98]
- **CLEVER** and **VERINA**. Benchmarks targeting verifier-in-the-loop code generation and proof-oriented programming tasks. [99, 100]
- **A benchmark for vericoding**, **AlgoVeri**, and **CASP**. Evaluation sets for formally verified program synthesis and code-level proof generation. [101, 102, 103]
- **DafnyBench** and **DafnyCOMP**. Benchmarks and competitions for LLM-assisted Dafny verification workflows. [104, 105]
- **VeriSoftBench**. Repository-scale Lean benchmark focused on software-verification obligations rather than pure mathematics. [106]
- **miniCodeProps**. Minimal Lean 4 benchmark for proving properties of small programs, useful for evaluating code-verification LLMs. [107]
- **Proving the Coding Interview**. Lean 4 benchmark built from interview-style algorithmic problems requiring formal verification proofs. [108]
- **Lean Workbook**. Large-scale dataset of Lean 4 problem statements collected from competition mathematics, intended to support training and evaluation of formal proving systems. [109]
- **Herald**. Benchmark for human-in-the-loop formal reasoning evaluation, measuring LLM ability to assist human provers on interactive theorem-proving tasks. [110]
- **miniCTX**. Context-dependent theorem proving benchmark for Lean that requires reasoning about file-level and project-level context beyond local tactic state. [111]

8 Adjacent specification and AI-native coding systems

- **Mog**. An AI-agent-oriented embedded language designed to be written by LLMs, with an explicit capability model and a compact language specification. [112]
- **CodeSpeak**. A spec-centric programming language and toolchain that generates conventional code from maintained specifications using LLMs. [113]
- **Spec Kit**. GitHub's open-source toolkit for spec-driven development with AI agents, centred on turning specifications into implementation plans and executable workflows. [114]

Bibliography

[1] Ke Weng et al. "Autoformalization in the Era of Large Language Models: A Survey". In: *CoRR* abs/2505.23486 (2025). URL: https://doi.org/10.48550/arXiv.2505.23486 [local PDF].

[2] Arshad Beg, Diarmuid P. O'Donoghue, and Rosemary Monahan. "A Short Survey on Formalising Software Requirements using Large Language Models". In: *CoRR* abs/2506.11874 (2025). URL: https://doi.org/10.48550/arXiv.2506.11874 [local PDF].

[3] Zhaoyu Li et al. "A Survey on Deep Learning for Theorem Proving". In: *CoRR* abs/2404.09939 (2024). URL: https://doi.org/10.48550/arXiv.2404.09939 [local PDF].

[4] Yedi Zhang et al. "The Fusion of Large Language Models and Formal Methods for Trustworthy AI Agents: A Roadmap". In: *CoRR* abs/2412.06512 (2024). URL: https://doi.org/10.48550/arXiv.2412.06512 [local PDF].

[5] Kaiyu Yang et al. "Position: Formal Mathematical Reasoning - A New Frontier in AI". In: *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025 - Position Paper Track*. OpenReview.net, 2025. URL: https://proceedings.mlr.press/v267/yang25az.html [local PDF].

[6] Alessio Ferrari and Paola Spoletini. "Formal requirements engineering and large language models: A two-way roadmap". In: *Inf. Softw. Technol.* 181 (2025), p. 107697. URL: https://doi.org/10.1016/j.infsof.2025.107697 [local PDF].

[7] *VERIFAI*. Project page. 2024. URL: https://www.cs.nuim.ie/research/pop/verifai.html [local PDF].

[8] Christopher Hahn et al. "Formal Specifications from Natural Language". In: *CoRR* abs/2206.01962 (2022). URL: https://doi.org/10.48550/arXiv.2206.01962 [local PDF].

[9] Yuhuai Wu et al. "Autoformalization with Large Language Models". In: *CoRR* abs/2205.12615 (2022). URL: https://doi.org/10.48550/arXiv.2205.12615 [local PDF].

[10] Jianqiao Lu et al. "Process-Driven Autoformalization in Lean 4". In: *CoRR* abs/2406.01940 (2024). URL: https://doi.org/10.48550/arXiv.2406.01940 [local PDF].

[11] Auguste Poiroux et al. "Improving Autoformalization using Type Checking". In: *CoRR* abs/2406.07222 (2024). URL: https://doi.org/10.48550/arXiv.2406.07222 [local PDF].

[12] Matthias Cosler et al. "nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models". In: *Computer Aided Verification - 35th International Conference, CAV 2023, Paris, France, July 17-22, 2023, Proceedings, Part II*. Ed. by Constantin Enea and Akash Lal. Lecture Notes in Computer Science. Springer, 2023, pp. 383–396. URL: https://doi.org/10.1007/978-3-031-37703-7%5C_18 [local PDF].

[13] Yongchao Chen et al. "NL2TL: Transforming Natural Languages to Temporal Logics using Large Language Models". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*. Ed. by Houda Bouamor, Juan Pino, and Kalika Bali. Association for Computational Linguistics, 2023, pp. 15880–15903. URL: https://doi.org/10.18653/v1/2023.emnlp-main.985 [local PDF].

[14] Lezhi Ma et al. "SpecGen: Automated Generation of Formal Program Specifications via Large Language Models". In: *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 2025, pp. 16–28. URL: https://doi.org/10.1109/ICSE55347.2025.00129 [local PDF].

[15] George Granberry, Wolfgang Ahrendt, and Moa Johansson. "Specify What? Enhancing Neural Specification Synthesis by Symbolic Methods". In: *Integrated Formal Methods - 19th International Conference, IFM 2024, Manchester, UK, November 13-15, 2024, Proceedings*. Ed. by Nikolai Kosmatov and Laura Kovács. Lecture Notes in Computer Science. Springer, 2024, pp. 307–325. URL: https://doi.org/10.1007/978-3-031-76554-4%5C_19 [local PDF].

[16] George Granberry, Wolfgang Ahrendt, and Moa Johansson. "Seeking Specifications: The Case for Neuro-Symbolic Specification Synthesis". In: *CoRR* abs/2504.21061 (2025). URL: https://doi.org/10.48550/arXiv.2504.21061 [local PDF].

[17] Shihao Xia et al. "Doc2Spec: Synthesizing Formal Programming Specifications from Natural Language via Grammar Induction". In: *CoRR* abs/2602.04892 (2026). URL: https://doi.org/10.48550/arXiv.2602.04892 [local PDF].

[18] Colin S. Gordon and Sergey Matskevich. "Trustworthy Formal Natural Language Specifications". In: *Proceedings of the 2023 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software, Onward! 2023, Cascais, Portugal, October 25-27, 2023*. Ed. by Tijs van der Storm and Robert Hirschfeld. ACM, 2023, pp. 50–70. URL: https://doi.org/10.1145/3622758.3622890 [local PDF].

[19] *Intent Formalization: A Grand Challenge for Reliable Coding in the Age of AI Agents*. Project / position page. 2025. URL: https://www.microsoft.com/en-us/research/publication/intent-formalization-a-grand-challenge-for-reliable-coding-in-the-age-of-ai-agents/ [local PDF].

[20] Madeline Endres et al. "Can Large Language Models Transform Natural Language Intent into Formal Method Postconditions?" In: *Proc. ACM Softw. Eng.* 1.FSE (2024), pp. 1889–1912. URL: https://doi.org/10.1145/3660791 [local PDF].

[21] Shuvendu K. Lahiri. "Evaluating LLM-driven User-Intent Formalization for Verification-Aware Languages". In: *Formal Methods in Computer-Aided Design, FMCAD 2024, Prague, Czech Republic, October 15-18, 2024*. Ed. by Nina Narodytska and Philipp Rümmer. IEEE, 2024, pp. 142–147. URL: https://doi.org/10.34727/2024/isbn.978-3-85448-065-5%5C_19 [local PDF].

[22] Weiqi Wang et al. "Supporting Software Formal Verification with Large Language Models: An Experimental Study". In: *CoRR* abs/2507.04857 (2025). URL: https://doi.org/10.48550/arXiv.2507.04857 [local PDF].

[23] Gabriel Poesia, Chloe Loughridge, and Nada Amin. "dafny-annotator: AI-Assisted Verification of Dafny Programs". In: *CoRR* abs/2411.15143 (2024). URL: https://doi.org/10.48550/arXiv.2411.15143 [local PDF].

[24] Md Rakib Hossain Misu et al. "Towards AI-Assisted Synthesis of Verified Dafny Methods". In: *Proc. ACM Softw. Eng.* 1.FSE (2024), pp. 812–835. URL: https://doi.org/10.1145/3643763.

[25] Eric Mugnier et al. "Laurel: Unblocking Automated Verification with Large Language Models". In: *Proc. ACM Program. Lang.* 9.OOPSLA1 (2025), pp. 1519–1545. URL: https://doi.org/10.1145/3720499 [local PDF].

[26] Marcos Cramer and Lucian McIntyre. "Verifying LLM-Generated Code in the Context of Software Verification with Ada/SPARK". In: *Proceedings of the 20th International Conference on Software Technologies, ICSOFT 2025, Bilbao, Spain, June 10-12, 2025*. Ed. by Massimo Mecella, Arend Rensink, and Leszek A. Maciaszek. SCITEPRESS, 2025, pp. 39–50. URL: https://doi.org/10.5220/0013461900003964 [local PDF].

[27] William Murphy et al. "Combining LLM Code Generation with Formal Specifications and Reactive Program Synthesis". In: *CoRR* abs/2410.19736 (2024). URL: https://doi.org/10.48550/arXiv.2410.19736 [local PDF].

[28] Merlijn Sevenhuijsen, Khashayar Etemadi, and Mattias Nyberg. "VeCoGen: Automating Generation of Formally Verified C Code With Large Language Models". In: *13th IEEE/ACM International Conference on Formal Methods in Software Engineering, FormaliSE@ICSE 2025, Ottawa, ON, Canada, April 27-28, 2025*. IEEE, 2025, pp. 101–112. URL: https://doi.org/10.1109/FormaliSE66629.2025.00017 [local PDF].

[29] João Pascoal Faria et al. "Automatic Generation of Formal Specification and Verification Annotations Using LLMs and Test Oracles". In: *CoRR* abs/2601.12845 (2026). URL: https://doi.org/10.48550/arXiv.2601.12845 [local PDF].

[30] Arshad Beg, Diarmuid P. O'Donoghue, and Rosemary Monahan. "Evaluating LLM-Generated ACSL Annotations for Formal Verification". In: *CoRR* abs/2602.13851 (2026). URL: https://doi.org/10.48550/arXiv.2602.13851 [local PDF].

[31] *AlphaVerus*. Project page. 2025. URL: https://alphaverus.github.io/ [local PDF].

[32] Sicheng Zhong et al. "RAG-Verus: Repository-Level Program Verification with LLMs using Retrieval Augmented Generation". In: *CoRR* abs/2502.05344 (2025). URL: https://doi.org/10.48550/arXiv.2502.05344 [local PDF].

[33] *Microsoft verus-proof-synthesis*. Repository / project page. 2025. URL: https://github.com/microsoft/verus-proof-synthesis [local PDF].

[34] Chenyuan Yang et al. "AutoVerus: Automated Proof Generation for Rust Code". In: *Proc. ACM Program. Lang.* 9.OOPSLA2 (2025), pp. 3454–3482. URL: https://doi.org/10.1145/3763174 [local PDF].

[35] Chenyuan Yang et al. "VeruSAGE: A Study of Agent-Based Verification for Rust Systems". In: *CoRR* abs/2512.18436 (2025). URL: https://doi.org/10.48550/arXiv.2512.18436 [local PDF].

[36] Chuyue Sun et al. "VeriStruct: AI-assisted Automated Verification of Data-Structure Modules in Verus". In: *CoRR* abs/2510.25015 (2025). URL: https://doi.org/10.48550/arXiv.2510.25015 [local PDF].

[37] *Generating Safety-Critical Automotive C-programs using LLMs with Formal Verification*. Joint industry-academic paper. 2025. URL: https://proceedings.mlr.press/v284/sevenhuijsen25a.html [local PDF].

[38] Chuyue Sun et al. "Clover: Closed-Loop Verifiable Code Generation". In: *AI Verification - First International Symposium, SAIV 2024, Montreal, QC, Canada, July 22-23, 2024, Proceedings*. Ed. by Guy Avni et al. Lecture Notes in Computer Science. Springer, 2024, pp. 134–155. URL: https://doi.org/10.1007/978-3-031-65112-0%5C_7 [local PDF].

[39] Tianyu Chen et al. "Automated Proof Generation for Rust Code via Self-Evolution". In: *CoRR* abs/2410.15756 (2024). URL: https://doi.org/10.48550/arXiv.2410.15756 [local PDF].

[40] Cheng Wen et al. "Enchanting Program Specification Synthesis by Large Language Models Using Static Analysis and Program Verification". In: *Computer Aided Verification - 36th International Conference, CAV 2024, Montreal, QC, Canada, July 24-27, 2024, Proceedings, Part II*. Lecture Notes in Computer Science. Springer, 2024, pp. 302–328. URL: https://doi.org/10.1007/978-3-031-65630-9%5C_16.

[41] Kyle Thompson et al. "Rango: Adaptive Retrieval-Augmented Proving for Automated Software Verification". In: *CoRR* abs/2412.14063 (2024). URL: https://doi.org/10.48550/arXiv.2412.14063 [local PDF].

[42] Saikat Chakraborty et al. "Towards Neural Synthesis for SMT-Assisted Proof-Oriented Programming". In: *CoRR* abs/2405.01787 (2024). URL: https://doi.org/10.48550/arXiv.2405.01787 [local PDF].

[43] Haoze Wu, Clark W. Barrett, and Nina Narodytska. "Lemur: Integrating Large Language Models in Automated Program Verification". In: *CoRR* abs/2310.04870 (2023). URL: https://doi.org/10.48550/arXiv.2310.04870 [local PDF].

[44] Adharsh Kamath et al. "Finding Inductive Loop Invariants using Large Language Models". In: *CoRR* abs/2311.07948 (2023). URL: https://doi.org/10.48550/arXiv.2311.07948 [local PDF].

[45] Marilyn Rego et al. "Evaluating the Ability of Large Language Models to Generate Verifiable Specifications in VeriFast". In: *CoRR* abs/2411.02318 (2024). URL: https://doi.org/10.48550/arXiv.2411.02318 [local PDF].

[46] Aleksandr Shefer et al. "Can LLMs Enable Verification in Mainstream Programming?" In: *CoRR* abs/2503.14183 (2025). URL: https://doi.org/10.48550/arXiv.2503.14183 [local PDF].

[47] Stanislas Polu and Ilya Sutskever. "Generative Language Modeling for Automated Theorem Proving". In: *CoRR* abs/2009.03393 (2020). URL: https://arxiv.org/abs/2009.03393 [local PDF].

[48] Albert Q. Jiang et al. "Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs". In: *CoRR* abs/2210.12283 (2022). URL: https://doi.org/10.48550/arXiv.2210.12283 [local PDF].

[49] Zhangir Azerbayev et al. "ProofNet: Autoformalizing and Formally Proving Undergraduate-Level Mathematics". In: *CoRR* abs/2302.12433 (2023). URL: https://doi.org/10.48550/arXiv.2302.12433 [local PDF].

[50] Emily First et al. "Baldur: Whole-Proof Generation and Repair with Large Language Models". In: *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3-9, 2023*. Ed. by Satish Chandra, Kelly Blincoe, and Paolo Tonella. ACM, 2023, pp. 1229–1241. URL: https://doi.org/10.1145/3611643.3616243 [local PDF].

[51] Kaiyu Yang et al. "LeanDojo: Theorem Proving with Retrieval-Augmented Language Models". In: *CoRR* abs/2306.15626 (2023). URL: https://doi.org/10.48550/arXiv.2306.15626 [local PDF].

[52] Amitayush Thakur, Yeming Wen, and Swarat Chaudhuri. "A Language-Agent Approach to Formal Theorem-Proving". In: *CoRR* abs/2310.04353 (2023). URL: https://doi.org/10.48550/arXiv.2310.04353 [local PDF].

[53] Huajian Xin et al. "DeepSeek-Prover: Advancing Theorem Proving in LLMs through Large-Scale Synthetic Data". In: *CoRR* abs/2405.14333 (2024). URL: https://doi.org/10.48550/arXiv.2405.14333 [local PDF].

[54] *DeepSeek-Prover-V1.5*. Repository / project page. 2025. URL: https://github.com/deepseek-ai/DeepSeek-Prover-V1.5 [local PDF].

[55] Xueliang Zhao et al. "SubgoalXL: Subgoal-based Expert Learning for Theorem Proving". In: *CoRR* abs/2408.11172 (2024). URL: https://doi.org/10.48550/arXiv.2408.11172 [local PDF].

[56] Haiming Wang et al. "LEGO-Prover: Neural Theorem Proving with Growing Libraries". In: *CoRR* abs/2310.00656 (2023). URL: https://doi.org/10.48550/arXiv.2310.00656 [local PDF].

[57] Peiyang Song, Kaiyu Yang, and Anima Anandkumar. "Lean Copilot: Large Language Models as Copilots for Theorem Proving in Lean". In: *International Conference on Neuro-symbolic Systems, 28-30 May 2025, University of Pennsylvania, Philadelphia, Pennsylvania, USA*. Ed. by George J. Pappas, Pradeep Ravikumar, and Sanjit A. Seshia. Proceedings of Machine Learning Research. PMLR, 2025, pp. 144–169. URL: https://proceedings.mlr.press/v288/song25a.html [local PDF].

[58] Azim Ospanov, Farzan Farnia, and Roozbeh Yousefzadeh. "APOLLO: Automated LLM and Lean Collaboration for Advanced Formal Reasoning". In: *CoRR* abs/2505.05758 (2025). URL: https://doi.org/10.48550/arXiv.2505.05758 [local PDF].

[59] Jilin Hu et al. "HybridProver: Augmenting Theorem Proving with LLM-Driven Proof Synthesis and Refinement". In: *CoRR* abs/2505.15740 (2025). URL: https://doi.org/10.48550/arXiv.2505.15740 [local PDF].

[60] Qiyuan Xu et al. *A Minimalist Proof Language for Neural Theorem Proving over Isabelle/HOL*. 2025. URL: https://arxiv.org/abs/2507.18885 [local PDF].

[61] Haoxin Tu et al. "Agentic Program Verification". In: *CoRR* abs/2511.17330 (2025). URL: https://doi.org/10.48550/arXiv.2511.17330 [local PDF].

[62] Jianyu Zhang et al. "Towards Real-World Industrial-Scale Verification: LLM-Driven Theorem Proving on seL4". In: *CoRR* abs/2602.08384 (2026). URL: https://doi.org/10.48550/arXiv.2602.08384 [local PDF].

[63] Zhe Hou. "Vibe Coding an LLM-powered Theorem Prover". In: *CoRR* abs/2601.04653 (2026). URL: https://doi.org/10.48550/arXiv.2601.04653 [local PDF].

[64] *Isabellm: A Playground for Free and Lightweight LLM-Guided Isabelle/HOL Provers*. Repository / project page. 2026. URL: https://github.com/zhehou/llm-isabelle [local PDF].

[65] David Yin and Jing Gao. "Generating Millions Of Lean Theorems With Proofs By Exploring State Transition Graphs". In: *CoRR* abs/2503.04772 (2025). URL: https://doi.org/10.48550/arXiv.2503.04772 [local PDF].

[66] *AI solves IMO problems at silver-medal level*. Google DeepMind blog post. 2024. URL: https://deepmind.google/blog/ai-solves-imo-problems-at-silver-medal-level/.

[67] *Olympiad-level formal mathematical reasoning with reinforcement learning*. Nature article. 2025. URL: https://www.nature.com/articles/s41586-025-09833-y.

[68] Tudor Achim et al. "Aristotle: IMO-level Automated Theorem Proving". In: *CoRR* abs/2510.01346 (2025). URL: https://doi.org/10.48550/arXiv.2510.01346 [local PDF].

[69] *Google DeepMind formal-conjectures*. Repository / project page. 2024. URL: https://github.com/google-deepmind/formal-conjectures.

[70] Ruida Wang et al. "TheoremLlama: Transforming General-Purpose LLMs into Lean4 Experts". In: *CoRR* abs/2407.03203 (2024). URL: https://doi.org/10.48550/arXiv.2407.03203 [local PDF].

[71] Xiaohan Lin et al. "FVEL: Interactive Formal Verification Environment with Large Language Models via Theorem Proving". In: *CoRR* abs/2406.14408 (2024). URL: https://doi.org/10.48550/arXiv.2406.14408 [local PDF].

[72] Lichen Zhang, Shuai Lu, and Nan Duan. "Selene: Pioneering Automated Proof in Software Verification". In: *CoRR* abs/2401.07663 (2024). URL: https://doi.org/10.48550/arXiv.2401.07663 [local PDF].

[73] Guillaume Lample et al. "HyperTree Proof Search for Neural Theorem Proving". In: *CoRR* abs/2205.11491 (2022). URL: https://doi.org/10.48550/arXiv.2205.11491 [local PDF].

[74] Haohan Lin et al. "Lean-STaR: Learning to Interleave Thinking and Proving". In: *CoRR* abs/2407.10040 (2024). URL: https://doi.org/10.48550/arXiv.2407.10040 [local PDF].

[75] Jin Peng Zhou et al. "REFACTOR: Learning to Extract Theorems from Proofs". In: *CoRR* abs/2402.17032 (2024). URL: https://doi.org/10.48550/arXiv.2402.17032 [local PDF].

[76] Jason Rute et al. "Graph2Tac: Learning Hierarchical Representations of Math Concepts in Theorem proving". In: *CoRR* abs/2401.02949 (2024). URL: https://doi.org/10.48550/arXiv.2401.02949 [local PDF].

[77] Stanislas Polu et al. "Formal Mathematics Statement Curriculum Learning". In: *CoRR* abs/2202.01344 (2022). URL: https://arxiv.org/abs/2202.01344 [local PDF].

[78] Zhangir Azerbayev et al. "Llemma: An Open Language Model For Mathematics". In: *CoRR* abs/2310.10631 (2023). URL: https://doi.org/10.48550/arXiv.2310.10631 [local PDF].

[79] Yong Lin et al. "Goedel-Prover: A Frontier Model for Open-Source Automated Theorem Proving". In: *CoRR* abs/2502.07640 (2025). URL: https://doi.org/10.48550/arXiv.2502.07640 [local PDF].

[80] Yong Lin et al. "Goedel-Prover-V2: Scaling Formal Theorem Proving with Scaffolded Data Synthesis and Self-Correction". In: *CoRR* abs/2508.03613 (2025). URL: https://doi.org/10.48550/arXiv.2508.03613 [local PDF].

[81] Z. Z. Ren et al. "DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition". In: *CoRR* abs/2504.21801 (2025). URL: https://doi.org/10.48550/arXiv.2504.21801 [local PDF].

[82] Haiming Wang et al. "Kimina-Prover Preview: Towards Large Formal Reasoning Models with Reinforcement Learning". In: *CoRR* abs/2504.11354 (2025). URL: https://doi.org/10.48550/arXiv.2504.11354 [local PDF].

[83] Trieu H. Trinh et al. "Solving olympiad geometry without human demonstrations". In: *Nat.* 625.7995 (2024), pp. 476–482. URL: https://doi.org/10.1038/s41586-023-06747-5.

[84] Logan Murphy et al. "Autoformalizing Euclidean Geometry". In: *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. Proceedings of Machine Learning Research. PMLR / OpenReview.net, 2024, pp. 36847–36893. URL: https://proceedings.mlr.press/v235/murphy24a.html.

[85] Theo Olausson et al. "LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*. Association for Computational Linguistics, 2023, pp. 5153–5176. URL: https://doi.org/10.18653/v1/2023.emnlp-main.313.

[86] Liangming Pan et al. "Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning". In: *Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, December 6-10, 2023*. Findings of ACL. Association for Computational Linguistics, 2023, pp. 3806–3824. URL: https://doi.org/10.18653/v1/2023.findings-emnlp.248.

[87] Xi Ye et al. "SatLM: Satisfiability-Aided Language Models Using Declarative Prompting". In: *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*. 2023. URL: http://papers.nips.cc/paper%5C_files/paper/2023/hash/8e9c7d4a48bdac81a58f983a64aaf42b-Abstract-Conference.html.

[88] Jesse Michael Han et al. "Proof Artifact Co-Training for Theorem Proving with Language Models". In: *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL: https://openreview.net/forum?id=rpxJc9j04U [local PDF].

[89] Chuanyang Zheng et al. "Lyra: Orchestrating Dual Correction in Automated Theorem Proving". In: *CoRR* abs/2309.15806 (2023). URL: https://doi.org/10.48550/arXiv.2309.15806 [local PDF].

[90] Xueliang Zhao, Wenda Li, and Lingpeng Kong. "Subgoal-based Demonstration Learning for Formal Theorem Proving". In: *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. Ed. by Ruslan Salakhutdinov et al. Proceedings of Machine Learning Research. PMLR / OpenReview.net, 2024, pp. 60832–60865. URL: https://proceedings.mlr.press/v235/zhao24h.html.

[91] Yang Li et al. "HUNYUANPROVER: A Scalable Data Synthesis Framework and Guided Tree Search for Automated Theorem Proving". In: *CoRR* abs/2412.20735 (2024). URL: https://doi.org/10.48550/arXiv.2412.20735 [local PDF].

[92] Haiming Wang et al. "Proving Theorems Recursively". In: *CoRR* abs/2405.14414 (2024). URL: https://doi.org/10.48550/arXiv.2405.14414 [local PDF].

[93] Ran Xin et al. "BFS-Prover: Scalable Best-First Tree Search for LLM-based Automatic Theorem Proving". In: *CoRR* abs/2502.03438 (2025). URL: https://doi.org/10.48550/arXiv.2502.03438 [local PDF].

[94] Zijian Wu et al. "InternLM2.5-StepProver: Advancing Automated Theorem Proving via Expert Iteration on Large-Scale LEAN Problems". In: *CoRR* abs/2410.15700 (2024). URL: https://doi.org/10.48550/arXiv.2410.15700 [local PDF].

[95] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. "miniF2F: a cross-system benchmark for formal Olympiad-level mathematics". In: *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL: https://openreview.net/forum?id=9ZPegFuFTFv [local PDF].

[96] George Tsoukalas et al. "PutnamBench: Evaluating Neural Theorem-Provers on the Putnam Mathematical Competition". In: *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*. Ed. by Amir Globersons et al. 2024. URL: http://papers.nips.cc/paper%5C_files/paper/2024/hash/1582eaf9e0cf349e1e5a6ee453100aa1-Abstract-Datasets%5C_and%5C_Benchmarks%5C_Track.html [local PDF].

[97] Zhouliang Yu et al. "FormalMATH: Benchmarking Formal Mathematical Reasoning of Large Language Models". In: *CoRR* abs/2505.02735 (2025). URL: https://doi.org/10.48550/arXiv.2505.02735 [local PDF].

[98] Chengwu Liu et al. "FIMO: A Challenge Formal Dataset for Automated Theorem Proving". In: *CoRR* abs/2309.04295 (2023). URL: https://doi.org/10.48550/arXiv.2309.04295 [local PDF].

[99] Amitayush Thakur et al. "CLEVER: A Curated Benchmark for Formally Verified Code Generation". In: *CoRR* abs/2505.13938 (2025). URL: https://doi.org/10.48550/arXiv.2505.13938 [local PDF].

[100] Zhe Ye et al. "VERINA: Benchmarking Verifiable Code Generation". In: *CoRR* abs/2505.23135 (2025). URL: https://doi.org/10.48550/arXiv.2505.23135 [local PDF].

[101] Sergiu Bursuc et al. "A benchmark for vericoding: formally verified program synthesis". In: *CoRR* abs/2509.22908 (2025). URL: https://doi.org/10.48550/arXiv.2509.22908 [local PDF].

[102] Haoyu Zhao et al. "AlgoVeri: An Aligned Benchmark for Verified Code Generation on Classical Algorithms". In: *CoRR* abs/2602.09464 (2026). URL: https://doi.org/10.48550/arXiv.2602.09464 [local PDF].

[103] Niclas Hertzberg et al. "CASP: An Evaluation Dataset for Formal Verification of C Code". In: *Bridging the Gap Between AI and Reality - Third International Conference on Bridging the Gap between AI and Reality, AISoLA 2025, Rhodes, Greece, November 1-5, 2025, Selected Papers*. Ed. by Bernhard Steffen. Lecture Notes in Computer Science. Springer, 2025, pp. 63–82. URL: https://doi.org/10.1007/978-3-032-07132-3%5C_5 [local PDF].

[104] Chloe Loughridge et al. "DafnyBench: A Benchmark for Formal Software Verification". In: *Trans. Mach. Learn. Res.* 2025 (2025). URL: https://openreview.net/forum?id=yBgTVWccIx [local PDF].

[105] Xu Xu et al. "Local Success Does Not Compose: Benchmarking Large Language Models for Compositional Formal Verification". In: *CoRR* abs/2509.23061 (2025). URL: https://doi.org/10.48550/arXiv.2509.23061 [local PDF].

[106] Yutong Xin et al. "VeriSoftBench: Repository-Scale Formal Verification Benchmarks for Lean". In: *CoRR* abs/2602.18307 (2026). URL: https://doi.org/10.48550/arXiv.2602.18307 [local PDF].

[107] Evan Lohn and Sean Welleck. "miniCodeProps: a Minimal Benchmark for Proving Code Properties". In: *CoRR* abs/2406.11915 (2024). URL: https://doi.org/10.48550/arXiv.2406.11915 [local PDF].

[108] Quinn Dougherty and Ronak Mehta. "Proving the Coding Interview: A Benchmark for Formally Verified Code Generation". In: *CoRR* abs/2502.05714 (2025). URL: https://doi.org/10.48550/arXiv.2502.05714 [local PDF].

[109] Huaiyuan Ying et al. "Lean Workbook: A large-scale Lean problem set formalized from natural language math problems". In: *CoRR* abs/2406.03847 (2024). URL: https://doi.org/10.48550/arXiv.2406.03847 [local PDF].

[110] Guoxiong Gao et al. "Herald: A Natural Language Annotated Lean 4 Dataset". In: *CoRR* abs/2410.10878 (2024). URL: https://doi.org/10.48550/arXiv.2410.10878 [local PDF].

[111] Jiewen Hu, Thomas Zhu, and Sean Welleck. "miniCTX: Neural Theorem Proving with (Long-)Contexts". In: *CoRR* abs/2408.03350 (2024). URL: https://doi.org/10.48550/arXiv.2408.03350 [local PDF].

[112] *Mog*. Project / language specification. 2026. URL: https://moglang.org/.

[113] *CodeSpeak*. Project / language page. 2026. URL: https://codespeak.dev/.

[114] *GitHub Spec Kit*. Repository / toolkit. 2025. URL: https://github.com/github/spec-kit.